

Project Proposal

Johnson Video Relational Database

Joshua Schwenk

Colorado State University Global

MIS407: Database Concepts

Dr. Shawn Harris

9/7/2025

Project Goals

Johnson Video Store has relied on stacks of invoices and piles of forms to manage purchasing, inventory, and customer rentals. This approach scatters critical information, makes returns hard to track, and prevents consistent calculations of fees. It also fails to preserve the distributor catalog data that influences pricing and format availability. The store rents multiple copies of the same movie across formats, sources titles from several distributors at different wholesale prices, and must keep both store-assigned identifiers and distributor identifiers for titles and items. Since the process is very labor intensive, Johnson Video Store had decided to implement a normalized relational database that centralized movies, distributor catalogs, inventory, customers, and rentals to improve accuracy and efficiency while promoting data-driven business decisions (Stair & Reynolds, 2018).

Business Rules

These rules translate Johnson Video Store's requirements into clear policies and constraints. The first set are policy rules the business cares about. The second set are schema rules already enforced by SQL.

Policy Rules

1. A customer may hold no more than one account.
2. A customer may have no more than three movies checked out at the same time.
3. A customer may not check out a VHS/DVD if their current overdue balance exceeds \$10.
4. A single copy of a movie may be rented to only one customer at a time.
5. The due date will be set for 7 –14 days after the rental date.
6. Late fees apply only when the return date is after the due date.
7. Rewind fees apply only to video/VHS formats.

Schema Rules

1. Each movie title is unique per release year (UNIQUE (title, release_year)).
2. Ratings must be one of: G, PG, PG-13, R, NC-17, Unrated (CHECK).
3. Only individuals recorded as MoviePerson may be designated as Actor or Director(FKs to MoviePerson).
4. Each Actor and Director pairing appears once (UNIQUE (movie_id, actor_id/director_id)).
5. Only Oscars are recorded (CHECK (award_name = 'Oscar')), and award_year must be between 1929-2125 (CHECK).
6. Distributor titles are unique per distributor (UNIQUE (distributor_id, distributor_movie_id)), and serials are unique per distributor (UNIQUE (distributor_id, distributor_serial_number)).
7. Every owned copy reference one Movie and status must be available, rented, lost, or damaged (CHECK).
8. A phone number is required (NOT NULL) and email is optional.
9. Rentals reference Inventory and Customer with ON DELETE RESTRICT to prevent accidental deletion of historical records.

The policies and schema rules define how the system should operate and what the database will generate. The policy rules guide day-to-day workflows, while schema rules enforce data integrity. The results are predictable operations, fewer errors, and reliable, data-driven decision-making.

Business Questions

This section outlines the key analytical questions the Johnson Video Store database should answer. The questions are grouped by stakeholder and decision area, and they map directly to the implemented schema so each can be addressed with SQL.

Distributor & Sourcing

1. Which distributor has Johnson Video not purchased from?

Purpose/Audience: Identify underutilized vendors for procurement to review and renegotiate; procurement.

2. Which distributor offers the largest selection of DVDs?

Purpose/Audience: Support sourcing strategy for dominant format; procurement.

3. For each internal movie, who is the lowest-cost distributor (by format)?

Purpose/Audience: Optimize replenishment costs; procurement.

Inventory & Catalog

4. Which Oscar-winning movies do we not currently own?

Purpose/Audience: Build targeted acquisition list aligned with demand; procurement.

5. Which titles currently have only one copy in inventory?

Purpose/Audience: Flag high-risk titles for replenishment planning to prevent stockouts and missed rentals; inventory and procurement.

Rentals

6. Which rentals were returned late and fees collected?

Purpose/Audience: Enforce policies, identify trends, and spot process issues; operations.

Marketing

7. Who were the top renters in the last 30 days?

Purpose/Audience: Target loyalty offers and retention campaigns; marketing.

8. Which locations have the most customers?

Purpose/Audience: Pinpoint high-density neighborhoods to prioritize billboards, promos, and in-store events; marketing.

9. What actors have the most credits among the titles we currently own?

Purpose/Audience: Plan themed promotions (e.g. Actor spotlights) using owned catalog; marketing.

10. Which people appear in the dataset as both actor and director?

Purpose/Audience: Curate special collections (e.g. Actor-Director) to engage film audiences.

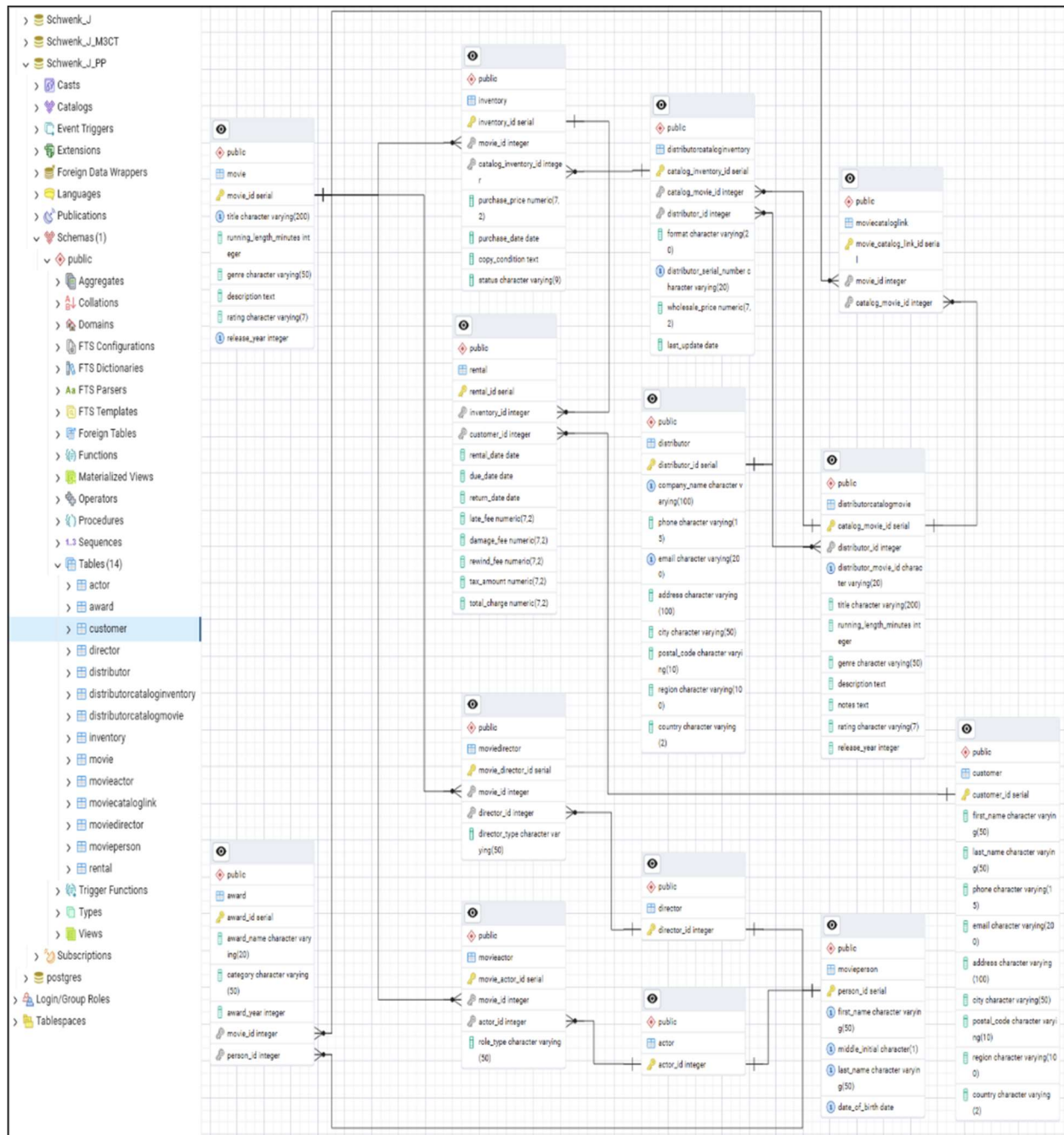
ERD

The ERD (Figure 1) was designed to replace manual records with an integrated schema. Following data modeling guidance, the model distinguishes movie titles from physical copies resulting in the elimination of the many-to-many (M:N) relationships with associative tables (e.g. MovieActor and MovieDirector). It

utilizes primary and foreign keys with CHECK and CONSTRAINT to preserve integrity and reduce redundancy (Mannino, 2022). It also records distributor catalog data at both the title and format which supports price comparisons during sourcing and simplifies future stock decisions. These choices align with systems analysis practices that begin with requirements, translating business rules in data structures, and ensures the schema supports both daily processing and data-driven decision-making (Valacich & George, 2017).

Figure 1

Johnson Video Store ERD



Note. Entity-relationship diagram for Johnson Video store using crow's foot notations to indicate cardinalities. Primary Keys (PK) and foreign keys (FK) are labeled with each entity. M:N relationships are solved with junction tables.

Metadata

The Attached Excel data dictionary lists each table with column definitions, keys, constraints, and example values. Below is a concise overview of the schema and why each table exists.

- **Movie** – One row per title + release year; enforces movie rating and sensible year.
- **MoviePerson** – Master people table (names, DOB) to avoid duplicates across roles.
- **Actor** – Subtype of MoviePerson; marks which people are actors.
- **MovieActor** – Junction for Movie <-> Actor (solves M:N casting); optional role_type.
- **Director** – Subtype of MoviePerson; marks which people are directors.
- **MovieDirector** – Junction for Movie <-> Director (solves M:N direction); optional director_type.
- **Award** – Oscar wins only; always tied to a Movie and optionally to a Person.
- **Distributor** – Supplier companies with unique contact info.
- **DistributorCatalogMovie** – Distributor's own title rows (their IDs, metadata).
- **MovieCatalogLink** – Maps internal Movie titles to distributor titles (even if not stocked) to support sourcing and price comparisons.
- **DistributorCatalogInventory** – Product number / format level (DVD/VHS) with distributor serial and wholesale price; allows multiple formats/prices per title and distributor.
- **Inventory** – Store-owned physical copies; tracks condition and status (available, rented, lost, damaged).
- **Customer** – Minimal PII for local renters; phone required, email optional.
- **Rental** – Transactions linking a customer to a specific copy with due/return dates and fees; deletes are restricted to preserve history.

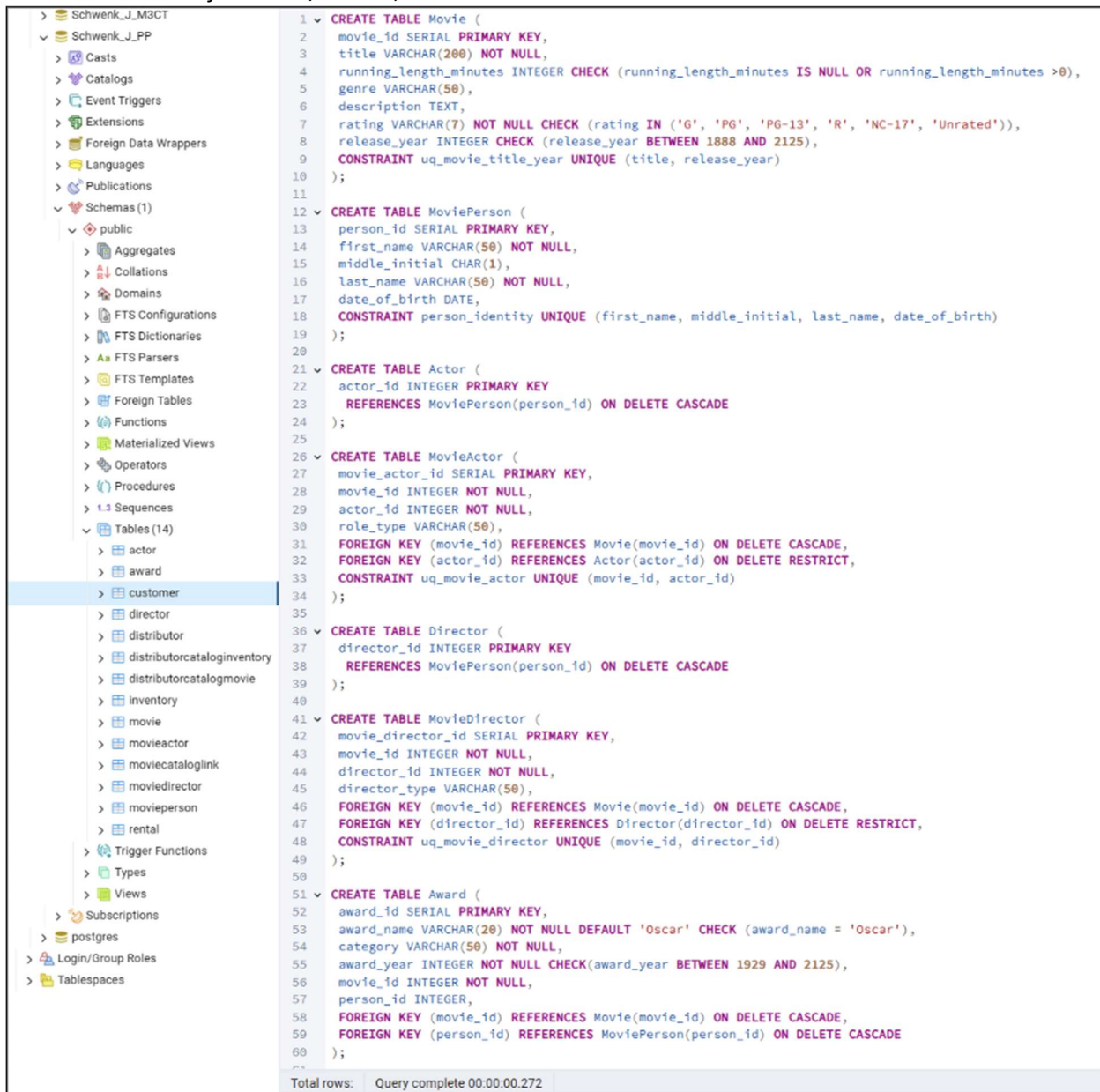
The schema separates titles from physical copies so many copies can exist per movie. It normalized people into single MoviePerson table with role subtypes (Actor, Director) to avoid duplication. M:N relationships are handled via junctions (MovieActor, MovieDirector). Distributor catalog is modeled independently (DistributorCatalogMovie, DistributorCatalogInventory) so the store can compare formats and wholesale prices even for titles that are not in stock. The MovieCatalogLink maps internal movies to external listing. Integrity is enforced with primary/foreign keys, uniqueness, and domain checks. Historical tables use RESTRICT DELETE to prevent rental and procurement history. This design reduces redundancy, keeps business rules explicit, and supports both day-to-day operations and sourcing/analytics.

SQL Code for Table Creation

The SQL (Figures 2-4) for table creation establishes a normalized foundation that replaces ad hoc paper processes with an integrated schema. The model resolves M:N relationships with associative entities, separating movie information from the physical inventory, preserves distributor identifiers for sourcing and price comparison, and enforces data quality with CONSTRAINTS and CHECKS. This structure supports day-to-day operations and enables analysis the store could not previously perform. The design also reflects system analysis practices by deriving data structures from business rules, so the database supports both operational processing and decision support (Valacich & George, 2017).

Figure 2

SQL Table Creation for Movie, Credits, & Awards



```

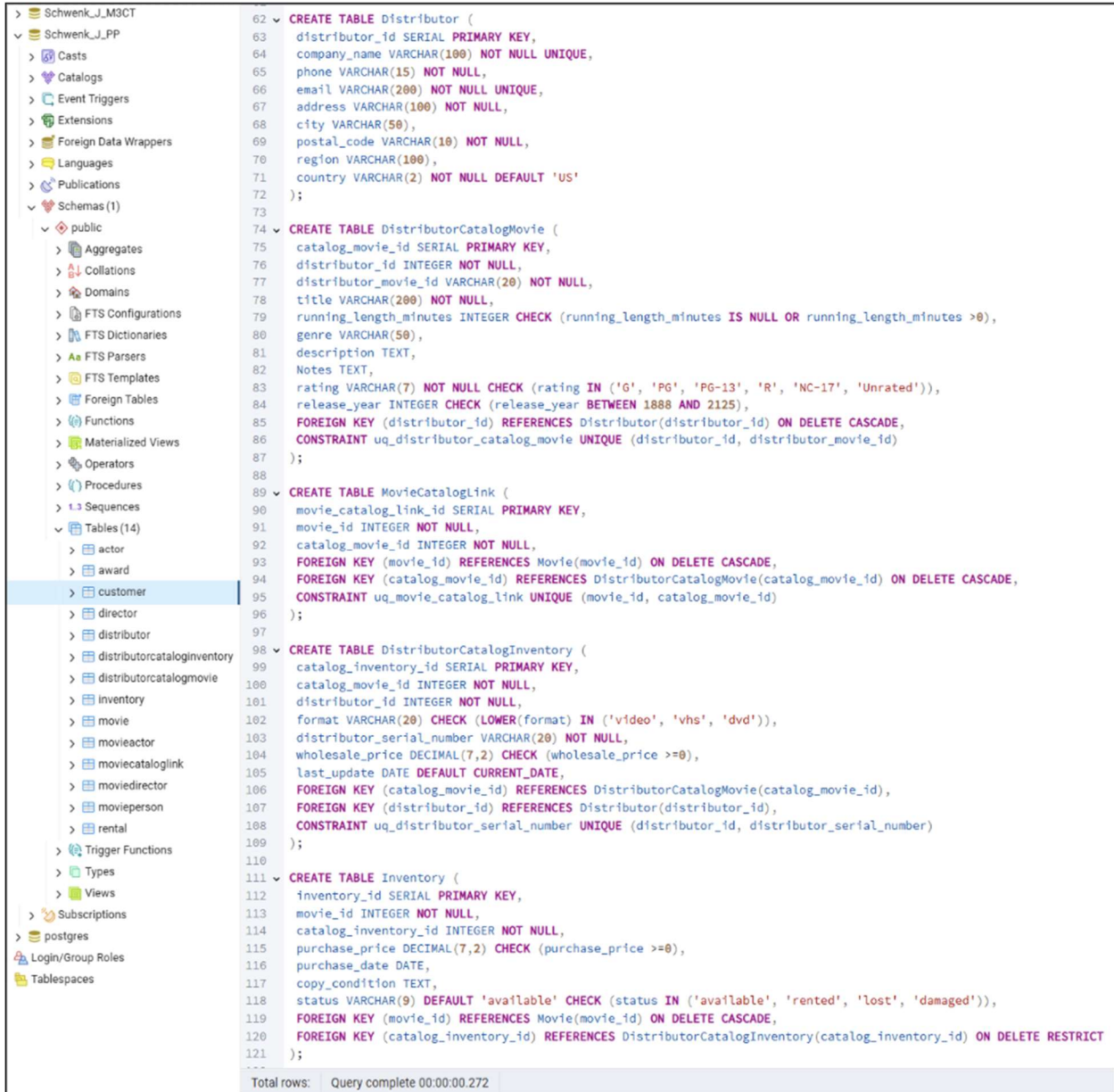
1 CREATE TABLE Movie (
2   movie_id SERIAL PRIMARY KEY,
3   title VARCHAR(200) NOT NULL,
4   running_length_minutes INTEGER CHECK (running_length_minutes IS NULL OR running_length_minutes > 0),
5   genre VARCHAR(50),
6   description TEXT,
7   rating VARCHAR(7) NOT NULL CHECK (rating IN ('G', 'PG', 'PG-13', 'R', 'NC-17', 'Unrated')),
8   release_year INTEGER CHECK (release_year BETWEEN 1888 AND 2125),
9   CONSTRAINT uq_movie_title_year UNIQUE (title, release_year)
10 );
11
12 CREATE TABLE MoviePerson (
13   person_id SERIAL PRIMARY KEY,
14   first_name VARCHAR(50) NOT NULL,
15   middle_initial CHAR(1),
16   last_name VARCHAR(50) NOT NULL,
17   date_of_birth DATE,
18   CONSTRAINT person_identity UNIQUE (first_name, middle_initial, last_name, date_of_birth)
19 );
20
21 CREATE TABLE Actor (
22   actor_id INTEGER PRIMARY KEY
23   REFERENCES MoviePerson(person_id) ON DELETE CASCADE
24 );
25
26 CREATE TABLE MovieActor (
27   movie_actor_id SERIAL PRIMARY KEY,
28   movie_id INTEGER NOT NULL,
29   actor_id INTEGER NOT NULL,
30   role_type VARCHAR(50),
31   FOREIGN KEY (movie_id) REFERENCES Movie(movie_id) ON DELETE CASCADE,
32   FOREIGN KEY (actor_id) REFERENCES Actor(actor_id) ON DELETE RESTRICT,
33   CONSTRAINT uq_movie_actor UNIQUE (movie_id, actor_id)
34 );
35
36 CREATE TABLE Director (
37   director_id INTEGER PRIMARY KEY
38   REFERENCES MoviePerson(person_id) ON DELETE CASCADE
39 );
40
41 CREATE TABLE MovieDirector (
42   movie_director_id SERIAL PRIMARY KEY,
43   movie_id INTEGER NOT NULL,
44   director_id INTEGER NOT NULL,
45   director_type VARCHAR(50),
46   FOREIGN KEY (movie_id) REFERENCES Movie(movie_id) ON DELETE CASCADE,
47   FOREIGN KEY (director_id) REFERENCES Director(director_id) ON DELETE RESTRICT,
48   CONSTRAINT uq_movie_director UNIQUE (movie_id, director_id)
49 );
50
51 CREATE TABLE Award (
52   award_id SERIAL PRIMARY KEY,
53   award_name VARCHAR(20) NOT NULL DEFAULT 'Oscar' CHECK (award_name = 'Oscar'),
54   category VARCHAR(50) NOT NULL,
55   award_year INTEGER NOT NULL CHECK (award_year BETWEEN 1929 AND 2125),
56   movie_id INTEGER NOT NULL,
57   person_id INTEGER,
58   FOREIGN KEY (movie_id) REFERENCES Movie(movie_id) ON DELETE CASCADE,
59   FOREIGN KEY (person_id) REFERENCES MoviePerson(person_id) ON DELETE CASCADE
60 );
61
Total rows: Query complete 00:00:00.272

```

Note. The SQL illustrates core movie and people structures. Movie enforces uniqueness by (title, release_year) and valid rating via a CHECK. MoviePerson is the master entity; Actor and Director are subtypes that share the same key. This is done to eliminate redundancies in data as a MoviePerson can be both an Actor and a Director. M:N is handled by junctions MovieActor and MovieDirector, each with UNIQUE command to prevent duplicate credits. Award is constrained to Oscars and may reference a Movie and/or a MoviePerson. Data types, foreign keys, and delete actions match the code shown.

Figure 3

SQL Table Creation for Distributor, DistributorCatalog, & Inventory



```

62 CREATE TABLE Distributor (
63     distributor_id SERIAL PRIMARY KEY,
64     company_name VARCHAR(100) NOT NULL UNIQUE,
65     phone VARCHAR(15) NOT NULL,
66     email VARCHAR(200) NOT NULL UNIQUE,
67     address VARCHAR(100) NOT NULL,
68     city VARCHAR(50),
69     postal_code VARCHAR(10) NOT NULL,
70     region VARCHAR(100),
71     country VARCHAR(2) NOT NULL DEFAULT 'US'
72 );
73
74 CREATE TABLE DistributorCatalogMovie (
75     catalog_movie_id SERIAL PRIMARY KEY,
76     distributor_id INTEGER NOT NULL,
77     distributor_movie_id VARCHAR(20) NOT NULL,
78     title VARCHAR(200) NOT NULL,
79     running_length_minutes INTEGER CHECK (running_length_minutes IS NULL OR running_length_minutes > 0),
80     genre VARCHAR(50),
81     description TEXT,
82     notes TEXT,
83     rating VARCHAR(7) NOT NULL CHECK (rating IN ('G', 'PG', 'PG-13', 'R', 'NC-17', 'Unrated')),
84     release_year INTEGER CHECK (release_year BETWEEN 1888 AND 2125),
85     FOREIGN KEY (distributor_id) REFERENCES Distributor(distributor_id) ON DELETE CASCADE,
86     CONSTRAINT uq_distributor_catalog_movie UNIQUE (distributor_id, distributor_movie_id)
87 );
88
89 CREATE TABLE MovieCatalogLink (
90     movie_catalog_link_id SERIAL PRIMARY KEY,
91     movie_id INTEGER NOT NULL,
92     catalog_movie_id INTEGER NOT NULL,
93     FOREIGN KEY (movie_id) REFERENCES Movie(movie_id) ON DELETE CASCADE,
94     FOREIGN KEY (catalog_movie_id) REFERENCES DistributorCatalogMovie(catalog_movie_id) ON DELETE CASCADE,
95     CONSTRAINT uq_movie_catalog_link UNIQUE (movie_id, catalog_movie_id)
96 );
97
98 CREATE TABLE DistributorCatalogInventory (
99     catalog_inventory_id SERIAL PRIMARY KEY,
100     catalog_movie_id INTEGER NOT NULL,
101     distributor_id INTEGER NOT NULL,
102     format VARCHAR(20) CHECK (LOWER(format) IN ('video', 'vhs', 'dvd')),
103     distributor_serial_number VARCHAR(20) NOT NULL,
104     wholesale_price DECIMAL(7,2) CHECK (wholesale_price >= 0),
105     last_update DATE DEFAULT CURRENT_DATE,
106     FOREIGN KEY (catalog_movie_id) REFERENCES DistributorCatalogMovie(catalog_movie_id),
107     FOREIGN KEY (distributor_id) REFERENCES Distributor(distributor_id),
108     CONSTRAINT uq_distributor_serial_number UNIQUE (distributor_id, distributor_serial_number)
109 );
110
111 CREATE TABLE Inventory (
112     inventory_id SERIAL PRIMARY KEY,
113     movie_id INTEGER NOT NULL,
114     catalog_inventory_id INTEGER NOT NULL,
115     purchase_price DECIMAL(7,2) CHECK (purchase_price >= 0),
116     purchase_date DATE,
117     copy_condition TEXT,
118     status VARCHAR(9) DEFAULT 'available' CHECK (status IN ('available', 'rented', 'lost', 'damaged')),
119     FOREIGN KEY (movie_id) REFERENCES Movie(movie_id) ON DELETE CASCADE,
120     FOREIGN KEY (catalog_inventory_id) REFERENCES DistributorCatalogInventory(catalog_inventory_id) ON DELETE RESTRICT
121 );

```

Total rows: Query complete 00:00:00.272

Note. SQL code for supplier and inventory structures. The distributor catalog is modeled separately to capture title & format level data independent of store inventory. This preserves distributor identifiers/pricing and supports prepurchase sourcing and analytics. DistributorCatalogMovie stores external IDs to avoid duplications. DistributorCatalogInventory stores format entries with serial numbers and wholesale price. The MovieCatalogLink table maps internal Movie rows to distributor titles even when no copies exist. This is done in order to enable videos/DVDs that are not stocked for reporting and a more streamlined procurement.

Figure 4

SQL Table Creation for Customer & Rentals

```

123 CREATE TABLE Customer (
124     customer_id SERIAL PRIMARY KEY,
125     first_name VARCHAR(50) NOT NULL,
126     last_name VARCHAR(50) NOT NULL,
127     phone VARCHAR(15) NOT NULL,
128     email VARCHAR(200),
129     address VARCHAR(100),
130     city VARCHAR(50),
131     postal_code VARCHAR(10),
132     region VARCHAR(100),
133     country VARCHAR(2) DEFAULT 'US'
134 );
135
136 CREATE TABLE Rental (
137     rental_id SERIAL PRIMARY KEY,
138     inventory_id INTEGER NOT NULL,
139     customer_id INTEGER NOT NULL,
140     rental_date DATE NOT NULL DEFAULT CURRENT_DATE,
141     due_date DATE NOT NULL CHECK (due_date >= rental_date),
142     return_date DATE CHECK (return_date IS NULL OR return_date >= rental_date),
143     late_fee DECIMAL(7,2) NOT NULL DEFAULT 0 CHECK (late_fee >= 0),
144     damage_fee DECIMAL(7,2) NOT NULL DEFAULT 0 CHECK (damage_fee >= 0),
145     rewind_fee DECIMAL(7,2) NOT NULL DEFAULT 0 CHECK (rewind_fee >= 0),
146     tax_amount DECIMAL(7,2) NOT NULL DEFAULT 0 CHECK (tax_amount >= 0),
147     total_charge DECIMAL(7,2) NOT NULL DEFAULT 0 CHECK (total_charge >= 0),
148     FOREIGN KEY (inventory_id) REFERENCES Inventory(inventory_id) ON DELETE RESTRICT,
149     FOREIGN KEY (customer_id) REFERENCES Customer(customer_id) ON DELETE RESTRICT
150 );

```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 272 msec.

Note. SQL code for customer and rental operations. Customer accounts require a phone number for late-return contact while keeping email optional for future programs. The Customer table is modeled as a separate entity rather than a subtype of a Person table to reflect distinct bounded contexts (operational customer management vs. filmographical information) and keep each model focused on its own policies and data needs (Evans, 2003). Rental REFERENCES Inventory and Customer via foreign keys with ON DELETE RESTRICT to preserve historical records. This structure supports core workflows (checkout, returns, fee assignments) and enables reporting on overdue items and repeat late customers.

Implementation Considerations

To operate the new database, it is recommended that:

1. A Database Administrator (DBA) will be assigned for technical operations (security, backups, performance, etc.) and a data steward for light governance (Mannino, 2022). This helps prevent outages and data loss while keeping data definitions consistent.
2. Task-focused training in pgAdmin 4 on lookups and rental/return workflows with short hands-on sessions and quick-reference guides. Communicate goals and benefits before rollout and run pilot/parallel period to reduce risk. Keep a rollback plan for initial release window (Stair & Reynolds, 2018). This improves adoption, reduces errors, and stabilizes day-to-day operations during rollout.
3. Conduct routine, basic operations hygiene. Maintain DEV/TEST/PROD separation, schedule nightly backups with periodic restore tests, enforce role-based permissions to protect customer/vendor information, and keep indexes and routine maintenance jobs in place for

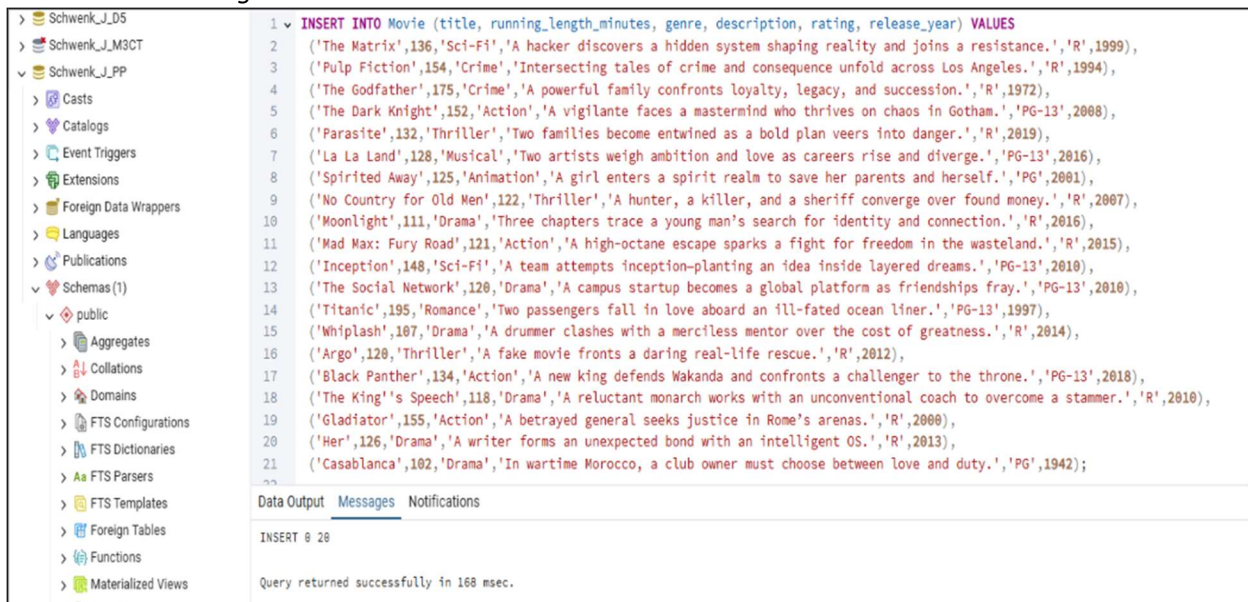
reliable performance. Structured testing across environments catches defects early and reduces deployment risks (Valacich & George, 2017).

Taken together, these practices turn the schema into a reliable operational database with clear ownership, recoverability, and guardrails for change. They establish a path for further development that protects data quality as the store grows. With this foundation, Johnson Video can add features such as loyalty programs and advanced reporting without disrupting existing workflows.

SQL Code for Database Seeding

The seeded dataset mixes award-winning and non-award movies, real-world directors and actors, and multiple distributors and formats to reflect typical operational testing. Together, the schema and seed data enable immediate validation of business rules and support early query for quality checks and reporting. This foundation positions Johnson Video Store to replace manual records with a consistent, auditable, and extensible data platform.

Figure 5
Movie Table Seeding



```

1  INSERT INTO Movie (title, running_length_minutes, genre, description, rating, release_year) VALUES
2  ('The Matrix',136,'Sci-Fi','A hacker discovers a hidden system shaping reality and joins a resistance.','R',1999),
3  ('Pulp Fiction',154,'Crime','Intersecting tales of crime and consequence unfold across Los Angeles.','R',1994),
4  ('The Godfather',175,'Crime','A powerful family confronts loyalty, legacy, and succession.','R',1972),
5  ('The Dark Knight',152,'Action','A vigilante faces a mastermind who thrives on chaos in Gotham.','PG-13',2008),
6  ('Parasite',132,'Thriller','Two families become entwined as a bold plan veers into danger.','R',2019),
7  ('La La Land',128,'Musical','Two artists weigh ambition and love as careers rise and diverge.','PG-13',2016),
8  ('Spirited Away',125,'Animation','A girl enters a spirit realm to save her parents and herself.','PG',2001),
9  ('No Country for Old Men',122,'Thriller','A hunter, a killer, and a sheriff converge over found money.','R',2007),
10 ('Moonlight',111,'Drama','Three chapters trace a young man's search for identity and connection.','R',2016),
11 ('Mad Max: Fury Road',121,'Action','A high-octane escape sparks a fight for freedom in the wasteland.','R',2015),
12 ('Inception',148,'Sci-Fi','A team attempts inception-planting an idea inside layered dreams.','PG-13',2010),
13 ('The Social Network',120,'Drama','A campus startup becomes a global platform as friendships fray.','PG-13',2010),
14 ('Titanic',195,'Romance','Two passengers fall in love aboard an ill-fated ocean liner.','PG-13',1997),
15 ('Whiplash',107,'Drama','A drummer clashes with a merciless mentor over the cost of greatness.','R',2014),
16 ('Argo',120,'Thriller','A fake movie fronts a daring real-life rescue.','R',2012),
17 ('Black Panther',134,'Action','A new king defends Wakanda and confronts a challenger to the throne.','PG-13',2018),
18 ('The King's Speech',118,'Drama','A reluctant monarch works with an unconventional coach to overcome a stammer.','R',2010),
19 ('Gladiator',155,'Action','A betrayed general seeks justice in Rome's arenas.','R',2000),
20 ('Her',126,'Drama','A writer forms an unexpected bond with an intelligent OS.','R',2013),
21 ('Casablanca',102,'Drama','In wartime Morocco, a club owner must choose between love and duty.','PG',1942);

```

Data Output Messages Notifications

INSERT 8 28

Query returned successfully in 168 msec.

Note. Screenshot of the INSERT INTO MoviePerson execution. These rows establish unique people used by both Actor and Director subtypes.

Figure 6

MoviePerson & Actor Table Seeding

The screenshot shows a database management interface with a schema tree on the left and a SQL editor on the right. The schema tree includes tables like actor, award, customer, director, distributor, distributorcataloginventory, distributorcatalogmovie, inventory, movie, movieactor, moviecataloglink, moviedirector, movieperson, rental, Trigger Functions, and Types. The SQL editor contains two insert statements:

```

23 INSERT INTO MoviePerson (first_name, middle_initial, last_name, date_of_birth) VALUES
24 ('Keanu', NULL, 'Reeves', '1964-09-02'),
25 ('Samuel', 'L', 'Jackson', '1948-12-21'),
26 ('Marlon', NULL, 'Brando', '1924-04-03'),
27 ('Christian', NULL, 'Bale', '1974-01-30'),
28 ('Kang-ho', NULL, 'Song', '1967-01-17'),
29 ('Emma', NULL, 'Stone', '1988-11-06'),
30 ('Rumi', NULL, 'Hiragi', '1987-08-01'),
31 ('Javier', NULL, 'Bardem', '1969-03-01'),
32 ('Mahershala', NULL, 'Ali', '1974-02-16'),
33 ('Charlize', NULL, 'Theron', '1975-08-07'),
34 ('Leonardo', NULL, 'DiCaprio', '1974-11-11'),
35 ('Jesse', NULL, 'Eisenberg', '1983-10-05'),
36 ('Kate', NULL, 'Winslet', '1975-10-05'),
37 ('J.K.', NULL, 'Simmons', '1955-01-09'),
38 ('Ben', NULL, 'Affleck', '1972-08-15'),
39 ('Michael', 'B', 'Jordan', '1969-02-09'),
40 ('Colin', NULL, 'Firth', '1960-09-10'),
41 ('Russell', NULL, 'Crowe', '1964-04-07'),
42 ('Joaquin', NULL, 'Phoenix', '1974-10-28'),
43 ('Humphrey', NULL, 'Bogart', '1899-12-25'),
44 ('Lana', NULL, 'Wachowski', '1965-06-21'),
45 ('Lilly', NULL, 'Wachowski', '1967-12-29'),
46 ('Quentin', NULL, 'Tarantino', '1963-03-27'),
47 ('Francis', 'F', 'Coppola', '1939-04-07'),
48 ('Christopher', NULL, 'Nolan', '1970-07-30'),
49 ('Bong', NULL, 'Joon-ho', '1969-09-14'),
50 ('Damien', NULL, 'Chazelle', '1985-01-19'),
51 ('Hayao', NULL, 'Miyazaki', '1941-01-05'),
52 ('Joel', NULL, 'Coen', '1954-11-29'),
53 ('Ethan', NULL, 'Coen', '1957-09-21'),
54 ('Barry', NULL, 'Jenkins', '1979-11-19'),
55 ('George', NULL, 'Miller', '1945-03-03'),
56 ('David', NULL, 'Fincher', '1962-08-28'),
57 ('James', NULL, 'Cameron', '1954-08-16'),
58 ('Ryan', NULL, 'Coogler', '1986-05-23'),
59 ('Tom', NULL, 'Hooper', '1972-10-05'),
60 ('Ridley', NULL, 'Scott', '1937-11-30'),
61 ('Spike', NULL, 'Jonze', '1969-10-22'),
62 ('Michael', NULL, 'Curtiz', '1886-12-24'),
63 ('Heath', NULL, 'Ledger', '1979-04-04');
64
65 INSERT INTO Actor (actor_id) VALUES
66 (1), (2), (3), (4), (5), (6), (7), (8), (9), (10), (11), (12), (13), (14), (15), (16), (17), (18), (19), (20), (40);
67

```

The bottom status bar shows: Data Output Messages Notifications. Query returned successfully in 168 msec.

Note. Screenshot of two insert blocks: INSERT INTO MoviePerson loads 20 individuals and INSERT INTO Actor designates a subset by reusing their person_id values as actor_id.

Figure 7

MovieActor, Director, & MovieDirector Table Seeding

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'public' schema is expanded, showing various database objects. Under 'Tables (14)', the 'customer' table is selected. The main query window contains the following SQL code:

```

69 INSERT INTO MovieActor (movie_id, actor_id, role_type) VALUES
70 (1,1,'lead'),
71 (2,2,'supporting'),
72 (3,3,'lead'),
73 (4,4,'lead'),
74 (5,5,'lead'),
75 (6,6,'lead'),
76 (7,7,'voice'),
77 (8,8,'lead'),
78 (9,9,'supporting'),
79 (10,10,'lead'),
80 (11,11,'lead'),
81 (12,12,'lead'),
82 (13,13,'lead'),
83 (14,14,'supporting'),
84 (15,15,'lead'),
85 (16,16,'supporting'),
86 (17,17,'lead'),
87 (18,18,'lead'),
88 (19,19,'lead'),
89 (20,20,'lead');
90
91 INSERT INTO Director (director_id) VALUES
92 (21),(22),(23),(24),(25),(26),(27),(28),(29),(30),(31),(32),(33),(34),(35),(36),(37),(38),(39),(15);
93
94 INSERT INTO MovieDirector (movie_id, director_id, director_type) VALUES
95 (1,21,'co-director'),
96 (1,22,'co-director'),
97 (2,23,'director'),
98 (3,24,'director'),
99 (4,25,'director'),
100 (5,26,'director'),
101 (6,27,'director'),
102 (7,28,'director'),
103 (8,29,'co-director'),
104 (8,30,'co-director'),
105 (9,31,'director'),
106 (10,32,'director'),
107 (11,25,'director'),
108 (12,33,'director'),
109 (13,34,'director'),
110 (14,27,'director'),
111 (15,15,'director'),
112 (16,35,'director'),
113 (17,36,'director'),
114 (18,37,'director'),
115 (19,38,'director'),
116 (20,39,'director');

```

At the bottom of the query window, the 'Messages' tab is active, showing the execution results:

```

Data Output Messages Notifications
INSERT 0 20
Query returned successfully in 168 msec.

```

Note. Screenshot of three insert blocks: INSERT INTO Director promotes selected people into director subtypes. INSERT INTO MovieActor links actors with a role_type. INSERT INTO MovieDirector records directing credits with optional director_type.

Figure 8

Award & Distributor Table Seeding

The screenshot displays a database client interface with a schema tree on the left, a SQL editor in the center, and a data output pane at the bottom.

Schema Tree (Left):

- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (14)
 - actor
 - award
 - customer
 - director
 - distributor
 - distributorcataloginventory
 - distributorcatalogmovie
 - inventory
 - movie
 - movieactor
 - moviecataloglink
 - moviedirector
 - movieperson
 - rental

SQL Editor (Center):

```

118 INSERT INTO Award (award_name, category, award_year, movie_id, person_id) VALUES
119 ('Oscar', 'Best Visual Effects', 2000, 1, NULL),
120 ('Oscar', 'Best Supporting Actor', 2009, 4, 40),
121 ('Oscar', 'Best Picture', 2020, 5, NULL),
122 ('Oscar', 'Best Director', 2017, 6, 27),
123 ('Oscar', 'Best Animated Feature', 2003, 7, NULL),
124 ('Oscar', 'Best Picture', 2008, 8, NULL),
125 ('Oscar', 'Best Picture', 2017, 9, NULL),
126 ('Oscar', 'Best Film Editing', 2016, 10, NULL),
127 ('Oscar', 'Best Cinematography', 2011, 11, NULL),
128 ('Oscar', 'Best Film Editing', 2011, 12, NULL),
129 ('Oscar', 'Best Picture', 1998, 13, NULL),
130 ('Oscar', 'Best Supporting Actor', 2015, 14, 14),
131 ('Oscar', 'Best Picture', 2013, 15, NULL),
132 ('Oscar', 'Best Original Score', 2019, 16, NULL),
133 ('Oscar', 'Best Actor', 2011, 17, 17),
134 ('Oscar', 'Best Actor', 2001, 18, 18),
135 ('Oscar', 'Best Original Screenplay', 2014, 19, 38),
136 ('Oscar', 'Best Picture', 1944, 20, NULL),
137 ('Oscar', 'Best Actor', 1973, 3, 3),
138 ('Oscar', 'Best Picture', 1973, 3, NULL);
139
140 INSERT INTO Distributor (company_name, phone, email, address, city, postal_code, region, country) VALUES
141 ('Distributor 01', '555-1000', 'contact@distributor01.com', '100 Main Ave', 'New York', '10004', 'NY', 'US'),
142 ('Distributor 02', '555-1001', 'contact@distributor02.com', '200 Main Ave', 'Los Angeles', '90026', 'CA', 'US'),
143 ('Distributor 03', '555-1002', 'contact@distributor03.com', '300 Main Ave', 'Chicago', '60601', 'IL', 'US'),
144 ('Distributor 04', '555-1003', 'contact@distributor04.com', '400 Main Ave', 'Austin', '78701', 'TX', 'US'),
145 ('Distributor 05', '555-1004', 'contact@distributor05.com', '500 Main Ave', 'Miami', '33132', 'FL', 'US'),
146 ('Distributor 06', '555-1005', 'contact@distributor06.com', '600 Main Ave', 'Seattle', '98101', 'WA', 'US'),
147 ('Distributor 07', '555-1006', 'contact@distributor07.com', '700 Main Ave', 'Boston', '02110', 'MA', 'US'),
148 ('Distributor 08', '555-1007', 'contact@distributor08.com', '800 Main Ave', 'Phoenix', '85004', 'AZ', 'US'),
149 ('Distributor 09', '555-1008', 'contact@distributor09.com', '900 Main Ave', 'Atlanta', '30303', 'GA', 'US'),
150 ('Distributor 10', '555-1009', 'contact@distributor10.com', '1000 Main Ave', 'San Francisco', '94105', 'CA', 'US'),
151 ('Distributor 11', '555-1010', 'contact@distributor11.com', '1100 Main Ave', 'Dallas', '75201', 'TX', 'US'),
152 ('Distributor 12', '555-1011', 'contact@distributor12.com', '1200 Main Ave', 'Minneapolis', '55402', 'MN', 'US'),
153 ('Distributor 13', '555-1012', 'contact@distributor13.com', '1300 Main Ave', 'Portland', '97204', 'OR', 'US'),
154 ('Distributor 14', '555-1013', 'contact@distributor14.com', '1400 Main Ave', 'Denver', '80202', 'CO', 'US'),
155 ('Distributor 15', '555-1014', 'contact@distributor15.com', '1500 Main Ave', 'Detroit', '48226', 'MI', 'US'),
156 ('Distributor 16', '555-1015', 'contact@distributor16.com', '1600 Main Ave', 'Philadelphia', '19103', 'PA', 'US'),
157 ('Distributor 17', '555-1016', 'contact@distributor17.com', '1700 Main Ave', 'Charlotte', '28202', 'NC', 'US'),
158 ('Distributor 18', '555-1017', 'contact@distributor18.com', '1800 Main Ave', 'Nashville', '37219', 'TN', 'US'),
159 ('Distributor 19', '555-1018', 'contact@distributor19.com', '1900 Main Ave', 'Salt Lake City', '84111', 'UT', 'US'),
160 ('Distributor 20', '555-1019', 'contact@distributor20.com', '2000 Main Ave', 'Kansas City', '64106', 'MO', 'US');
  
```

Data Output (Bottom):

INSERT 0 20

Query returned successfully in 168 msec.

Note. Screenshot of two insert blocks: *INSERT INTO Award* loads records Academy Awards only and ties each row to a film. *INSERT INTO Distributor* load creates supplier records and standardized contact fields.

Figure 9

DistributorCatalogMovie & MovieCatalogLink Table Seeding

Schwenk_UDS	162	INSERT INTO DistributorCatalogMovie (distributor_id, distributor_movie_id, title, running_length_minutes, genre, description, Notes, rating, release_year) VALUES
Schwenk_UDMCT	163	(1, 'MATRIX-01', 'The Matrix', 136, 'Sci-Fi', 'A hacker discovers a hidden system shaping reality and joins a resistance.', 'Street date 2025-09-01; sci-fi bundle eligible.', 'R', 1999),
Schwenk_UDPP	164	(2, 'PULP-94-02', 'Pulp Fiction', 154, 'Crime', 'Intersecting tales of crime and consequence unfold across Los Angeles.', 'Back-catalog evergreen; VHS limited.', 'R', 1994),
	165	(3, 'GODF-72-03', 'The Godfather', 175, 'Crime', 'A powerful family confronts loyalty, legacy, and succession.', '4K remaster Q4; extra lead time.', 'R', 1972),
> Casts	166	(4, 'TDR-88-04', 'The Dark Knight', 152, 'Action', 'A vigilante faces a mastermind who thrives on chaos in Gotham.', 'Strict street-date compliance.', 'PG-13', 2008),
> Catalogs	167	(5, 'PARA-19-05', 'Parasite', 132, 'Thriller', 'Two families become entwined as a bold plan veers into danger.', 'Bilingual packaging; add subtitle sticker.', 'R', 2019),
> Event Triggers	168	(6, 'LALA-16-06', 'La La Land', 128, 'Musical', 'Two artists weigh ambition and love as careers diverge.', 'Musical bundle rebate eligible.', 'PG-13', 2016),
> Extensions	169	(7, 'SPIR-01-07', 'Spirited Away', 125, 'Animation', 'A girl enters a spirit realm to save her parents.', 'High family demand; 3P audio + ENG/SPA subs.', 'PG', 2001),
> Foreign Data Wrappers	170	(8, 'NOC-97-08', 'No Country for Old Men', 122, 'Thriller', 'A hunter, a killer, and a sheriff converge over found money.', 'Limited restock-preorder advised.', 'R', 2007),
> Languages	171	(9, 'MOON-16-09', 'Moonlight', 111, 'Drama', 'Three chapters trace a young man's search for identity and connection.', 'Slipcover variant this quarter.', 'R', 2016),
> Publications	172	(10, 'FURY-15-10', 'Mad Max: Fury Road', 121, 'Action', 'A high-octane escape sparks a fight for freedom.', 'Steelbook on backorder.', 'R', 2015),
> Publications	173	(11, 'INC-10-11', 'Inception', 148, 'Sci-Fi', 'A team attempts inception-planting an idea inside layered dreams.', 'Pairs with mind-bender promo.', 'PG-13', 2010),
> Schemas (1)	174	(13, 'SOC-18-13', 'The Social Network', 120, 'Drama', 'A campus startup becomes a global platform as friendships fray.', 'Campus tie-in available.', 'PG-13', 2010),
> public	175	(14, 'TIT-97-14', 'Titanic', 195, 'Romance', 'Two passengers fall in love aboard an ill-fated ocean liner.', 'Seasonal bump near Valentine's Day.', 'PG-13', 1997),
> Aggregates	176	(16, 'WHIP-14-16', 'Whiplash', 107, 'Drama', 'A drummer clashes with a merciless mentor over greatness.', 'Jazz series cross-promo.', 'R', 2014),
> Collations	177	(17, 'ARGO-12-17', 'Argo', 120, 'Thriller', 'A fake movie fronts a daring real-life rescue.', 'Awards sticker pack included.', 'R', 2012),
> Domains	178	(18, 'BP-18-18', 'Black Panther', 134, 'Action', 'A king defends Wakanda while facing a challenger.', 'High demand; weekly replenishment.', 'PG-13', 2018),
> FTS Configurations	179	(20, 'TKS-18-20', 'The King's Speech', 118, 'Drama', 'A monarch works to overcome a stammer with an unconventional coach.', 'OSCAR label rights cleared.', 'R', 2010),
> FTS Dictionaries	180	(3, 'GLAD-00-03', 'Gladiator', 155, 'Action', 'A betrayed general seeks justice in Rome's arenas.', 'Anniversary edition; preorder open.', 'R', 2000),
> FTS Parsers	181	(10, 'HER-13-10', 'Her', 126, 'Drama', 'A writer forms an unexpected bond with an intelligent OS.', 'Eco-case; lightweight cartons.', 'R', 2013),
> FTS Templates	182	(20, 'CASA-42-20', 'Casablanca', 102, 'Drama', 'In wartime Morocco, a club owner chooses between love and duty.', 'Holiday demand spike; plan stock.', 'PG', 1942),
> Foreign Tables	183	(10, 'MATRIX-10', 'The Matrix', 136, 'Sci-Fi', 'A hacker discovers a hidden system shaping reality and joins a resistance.', 'Alt vendor; see metal-case variant.', 'R', 1999),
> Functions	184	(9, 'PULP-94-09', 'Pulp Fiction', 154, 'Crime', 'Intersecting tales of crime and consequence unfold across Los Angeles.', 'Alt vendor; bundle with Tarantino set.', 'R', 1994),
> Materialized Views	185	(13, 'PARA-19-13', 'Parasite', 132, 'Thriller', 'Two families become entwined as a bold plan veers into danger.', 'Alt vendor; bilingual slipcover.', 'R', 2019),
> Operators	186	(11, 'NOC-97-11', 'No Country for Old Men', 122, 'Thriller', 'A hunter, a killer, and a sheriff converge over found money.', 'Alt vendor; early ship window.', 'R', 2007);
> Procedures	187	
> Sequences	188	INSERT INTO MovieCatalogLink (movie_id, catalog_movie_id) VALUES
> Tables (14)	189	(1,1), (2,2), (3,3), (4,4), (5,5), (6,6), (7,7), (8,8), (9,9), (10,10), (11,11), (12,12), (13,13), (14,14), (15,15), (16,16), (17,17), (18,18), (19,19), (20,20);
> actor	190	
> award	191	
		Data Output Messages Notifications
		INSERT @ 20
		Query returned successfully in 168 msec.

Note. Screenshot of two insert blocks: INSERT INTO DistributorCatalogMovie loads distributor-provided title records including each vendor's external distributor_movie_id, synopsis, rating release year, and operational notes. INSERT INTO MovieCatalogLink maps internal movie rows to those catalog titles.

Figure 10

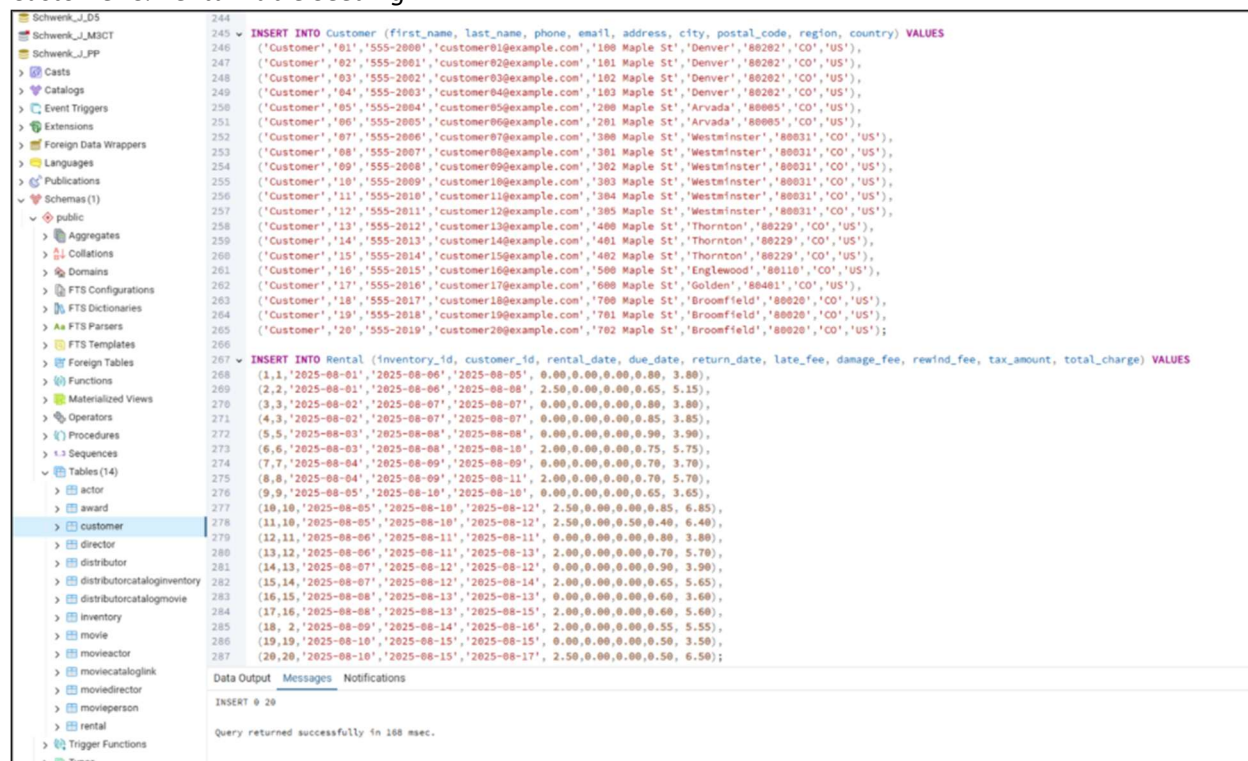
DistributorCatalogInventory & Inventory Table Seeding

Schwenk_UDS	191	INSERT INTO DistributorCatalogInventory (catalog_movie_id, distributor_id, format, distributor_serial_number, wholesale_price, last_update) VALUES
Schwenk_UDMCT	192	(1,1, 'dvd', 'D01-MATRIX-DVD', 9.99, '2025-07-01'),
Schwenk_UDPP	193	(2,2, 'vhs', 'D02-PULP-VHS', 6.25, '2025-07-01'),
> Casts	194	(3,3, 'dvd', 'D03-GODF-DVD', 11.25, '2025-07-02'),
> Catalogs	195	(4,4, 'dvd', 'D04-TDR-DVD', 10.50, '2025-07-02'),
> Event Triggers	196	(5,5, 'dvd', 'D05-PARA-DVD', 12.75, '2025-07-03'),
> Extensions	197	(6,6, 'dvd', 'D06-LALA-DVD', 10.10, '2025-07-04'),
> Foreign Data Wrappers	198	(7,7, 'dvd', 'D07-SPIR-DVD', 9.20, '2025-07-05'),
> Languages	199	(8,8, 'dvd', 'D08-NOC-DVD', 10.80, '2025-07-05'),
> Publications	200	(9,9, 'dvd', 'D09-MOON-DVD', 9.85, '2025-07-06'),
> Schemas (1)	201	(10,10, 'dvd', 'D10-FURY-DVD', 11.99, '2025-07-06'),
> public	202	(11,11, 'dvd', 'D11-INC-DVD', 10.75, '2025-07-07'),
> Aggregates	203	(12,12, 'dvd', 'D12-SOC-DVD', 9.60, '2025-07-07'),
> Collations	204	(13,13, 'dvd', 'D13-TIT-DVD', 12.40, '2025-07-08'),
> Domains	205	(14,14, 'dvd', 'D14-WHIP-DVD', 8.90, '2025-07-08'),
> FTS Configurations	206	(15,15, 'dvd', 'D15-ARGO-DVD', 9.70, '2025-07-09'),
> FTS Dictionaries	207	(16,16, 'dvd', 'D16-BP-DVD', 10.99, '2025-07-09'),
> FTS Parsers	208	(17,17, 'dvd', 'D17-TKS-DVD', 9.30, '2025-07-10'),
> FTS Templates	209	(18,18, 'dvd', 'D18-GLAD-DVD', 9.95, '2025-07-10'),
> Foreign Tables	210	(19,19, 'dvd', 'D19-HER-DVD', 11.15, '2025-07-11'),
> Functions	211	(20,20, 'vhs', 'D20-CASA-VHS', 5.95, '2025-07-11'),
> Materialized Views	212	(2,2, 'dvd', 'D02-PULP-DVD', 9.10, '2025-07-12'),
> Operators	213	(1,10, 'dvd', 'D10-MATRIX-DVD', 10.49, '2025-07-12'),
> Procedures	214	(1,10, 'vhs', 'D10-MATRIX-VHS', 6.10, '2025-07-12'),
> Sequences	215	(8,11, 'dvd', 'D11-NOC-DVD', 10.20, '2025-07-12'),
> Tables (14)	216	(5,13, 'dvd', 'D13-PARA-DVD', 12.10, '2025-07-12'),
> actor	217	(10,10, 'vhs', 'D10-FURY-VHS', 6.75, '2025-07-12'),
> award	218	(20,20, 'dvd', 'D20-CASA-DVD', 8.40, '2025-07-12'),
> customer	219	(14,16, 'vhs', 'D16-WHIP-VHS', 5.60, '2025-07-12'),
> director	220	(13,14, 'vhs', 'D14-TIT-VHS', 6.90, '2025-07-12'),
> distributor	221	(7,7, 'vhs', 'D07-SPIR-VHS', 6.20, '2025-07-12');
> distributorcataloginventory	222	
> distributorcatalogmovie	223	INSERT INTO Inventory (movie_id, catalog_inventory_id, purchase_price, purchase_date, copy_condition, status) VALUES
> inventory	224	(1,1,14.99, '2025-07-12', 'Good', 'available'),
> movie	225	(1,22,15.49, '2025-07-12', 'Good', 'available'),
> movieactor	226	(2,21,12.99, '2025-07-12', 'Fair', 'available'),
> moviecataloglink	227	(3,3,15.49, '2025-07-12', 'Poor', 'available'),
> moviedirector	228	(4,4,14.25, '2025-07-12', 'Good', 'available'),
> movieperson	229	(6,6,14.10, '2025-07-12', 'Good', 'available'),
> rental	230	(7,7,13.90, '2025-07-12', 'Good', 'available'),
> Trigger Functions	231	(8,8,14.60, '2025-07-12', 'Good', 'available'),
> Types	232	(6,24,14.20, '2025-07-12', 'Good', 'available'),
> Views	233	(10,10,15.99, '2025-07-12', 'Good', 'available'),
> Subscriptions	234	(10,26,7.25, '2025-07-12', 'Good', 'available'),
> postgres	235	(11,11,14.75, '2025-07-12', 'Good', 'available'),
> Login/Group Roles	236	(12,12,13.60, '2025-07-12', 'Poor', 'available'),
> Tablespaces	237	(13,13,16.40, '2025-07-12', 'Good', 'available'),
	238	(14,14,12.90, '2025-07-12', 'Fair', 'available'),
	239	(15,15,13.70, '2025-07-12', 'Good', 'available'),
	240	(16,16,14.90, '2025-07-12', 'Good', 'available'),
	241	(17,17,13.30, '2025-07-12', 'Poor', 'available'),
	242	(18,18,13.95, '2025-07-12', 'Good', 'available'),
	243	(19,19,15.15, '2025-07-12', 'Fair', 'available');
	244	
		Data Output Messages Notifications
		INSERT @ 20
		Query returned successfully in 168 msec.

Note. Screenshot of two insert blocks: INSERT INTO DistributorCatalogInventory loads distributor product codes at the formal level (DVD/VHS). INSERT INTO Inventory creates store-owned copies linked to Movie and the catalog product code.

Figure 11

Customer & Rental Table Seeding



Note. Screenshot of two insert blocks: INSERT INTO Customer loads Denver-Metro accounts. INSERT INTO RENTAL records transactions that reference Inventory and Customer via foreign keys.

Answers to Business Questions

This section presents concise, query backed answers to the key business questions defined earlier. Each item includes the SQL used against the seeded PostgreSQL database.

Distributor & Sourcing

Which distributor has Johnson Video not purchased from?

Figure 12

Distributors with No Purchases

The screenshot shows a database management interface. On the left is a tree view of database objects, with 'Schwenk_J_PP' selected. The main area displays a SQL query:

```

1 SELECT d.distributor_id, d.company_name
2 FROM Distributor AS d
3 LEFT JOIN DistributorCatalogInventory AS dci
4     ON dci.distributor_id = d.distributor_id
5 LEFT JOIN Inventory AS i
6     ON i.catalog_inventory_id = dci.catalog_inventory_id
7 GROUP BY d.distributor_id, d.company_name
8 HAVING COUNT(i.inventory_id) = 0
9 ORDER BY d.company_name;

```

Below the query, the 'Data Output' tab is active, showing a table with 5 rows of results:

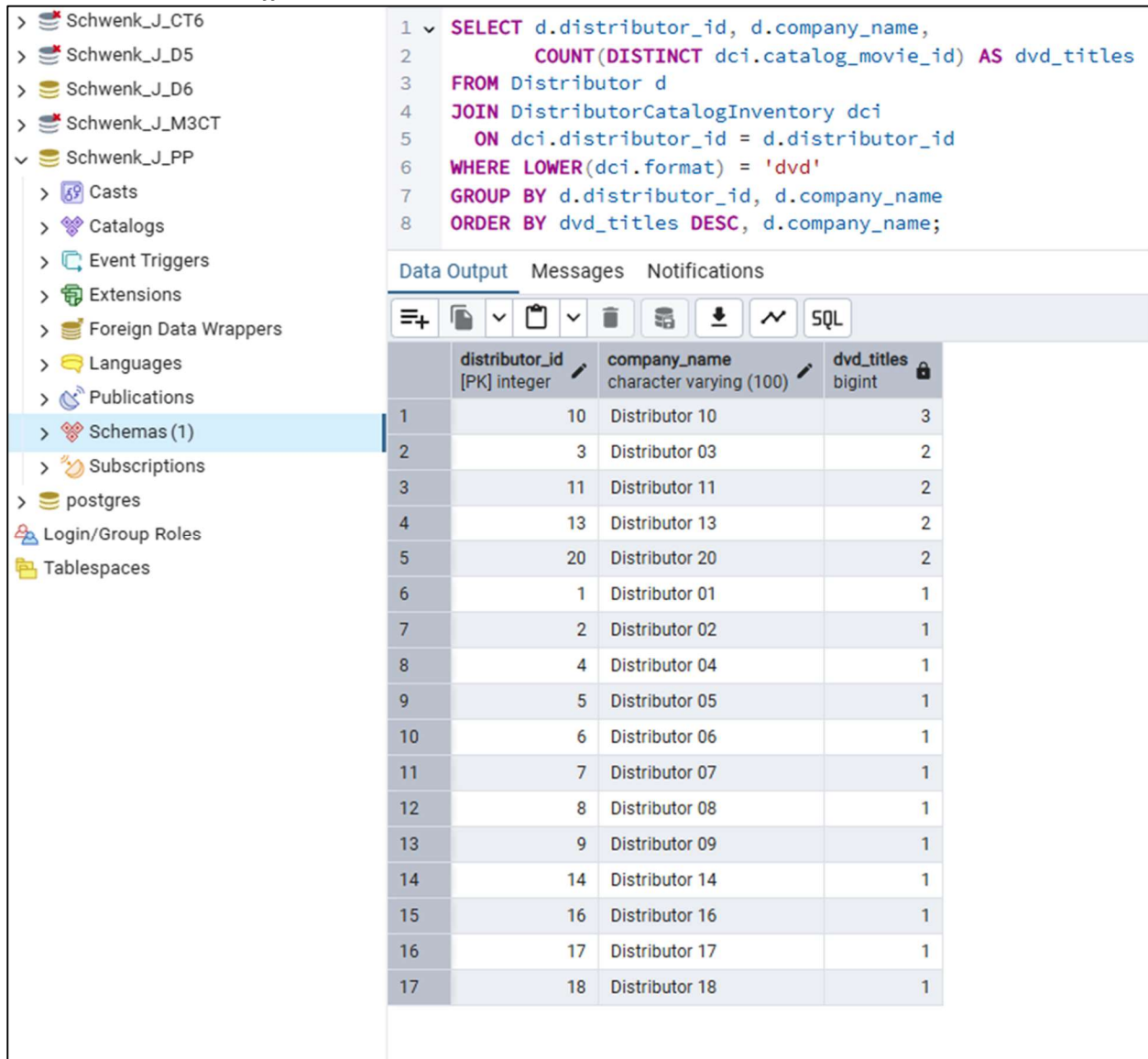
	distributor_id [PK] integer	company_name character varying (100)
1	5	Distributor 05
2	9	Distributor 09
3	12	Distributor 12
4	15	Distributor 15
5	19	Distributor 19

Note. Query identifies distributors from whom Johnson Video has not purchased any product. Implementation uses a LEFT JOIN from Distributor -> DistributorCatalogInventory -> Inventory, grouping by distributor and filtering with HAVING COUNT(i.inventory_id) = 0. "Purchased" is operationalized as the presence of at least one Inventory row referencing a distributor SKU

Which distributor offers the largest selection of DVDs?

Figure 13

Distributor DVD Titles Offered



The screenshot displays a database management interface. On the left is a tree view of the database schema, including catalogs, event triggers, extensions, foreign data wrappers, languages, publications, schemas (1), subscriptions, and tablespaces. The main area shows a SQL query and its output.

SQL Query:

```

1 SELECT d.distributor_id, d.company_name,
2       COUNT(DISTINCT dci.catalog_movie_id) AS dvd_titles
3 FROM Distributor d
4 JOIN DistributorCatalogInventory dci
5     ON dci.distributor_id = d.distributor_id
6 WHERE LOWER(dci.format) = 'dvd'
7 GROUP BY d.distributor_id, d.company_name
8 ORDER BY dvd_titles DESC, d.company_name;

```

Data Output:

	distributor_id [PK] integer	company_name character varying (100)	dvd_titles bigint
1	10	Distributor 10	3
2	3	Distributor 03	2
3	11	Distributor 11	2
4	13	Distributor 13	2
5	20	Distributor 20	2
6	1	Distributor 01	1
7	2	Distributor 02	1
8	4	Distributor 04	1
9	5	Distributor 05	1
10	6	Distributor 06	1
11	7	Distributor 07	1
12	8	Distributor 08	1
13	9	Distributor 09	1
14	14	Distributor 14	1
15	16	Distributor 16	1
16	17	Distributor 17	1
17	18	Distributor 18	1

Note. Query ranks distributors by the number of distinct DVD titles they offer. Implementation joins Distributor -> DistributorCatalogInventory, filters LOWER(format) = 'dvd', and aggregates with COUNT(DISTINCT dci.catalog_movie_id) per distributor. The results are ordered descending. Using DISTINCT avoids double-counting multiple DVD's for the same title.

For each internal movie, who is the lowest-cost distributor (by format)?

Figure 14

Lowest-Cost Distributor per Movie & Format

Schwenk_J_CT6

Schwenk_J_D5

Schwenk_J_D6

Schwenk_J_M3CT

Schwenk_J_PP

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas (1)

Subscriptions

postgres

Login/Group Roles

Tablespaces

WITH price_rows AS (

SELECT m.movie_id, m.title,

LOWER(dci.format) AS format,

d.distributor_id, d.company_name,

dci.wholesale_price,

ROW_NUMBER() OVER (

PARTITION BY m.movie_id, LOWER(dci.format)

ORDER BY dci.wholesale_price ASC, d.company_name

AS rn

FROM Movie m

JOIN MovieCatalogLink mcl

ON mcl.movie_id = m.movie_id

JOIN DistributorCatalogMovie dcm

ON dcm.catalog_movie_id = mcl.catalog_movie_id

JOIN DistributorCatalogInventory dci

ON dci.catalog_movie_id = dcm.catalog_movie_id

JOIN Distributor d

ON d.distributor_id = dci.distributor_id

)

SELECT title, format, company_name AS best_distributor, wholesale_price AS best_wholesale

FROM price_rows

WHERE rn = 1

ORDER BY title, format;

Data Output

Messages

Notifications

<

Note. Query maps internal titles (Movie) to distributor listings via MovieCatalogLink -> DistributorCatalogMovie -> DistributorCatalogInventory, joins Distributor for names, and then selects the minimum wholesale price per (movie, format). Implementation uses a window function (ROW_NUMBER() partitioned by movie and LOWER(format)) to keep the cheapest row. Prices reflect catalog offers (not necessarily owned copies in Inventory).

Inventory & Catalog

Which Oscar-winning movies do we not currently own?

Figure 15

Oscar-Winning Titles Not Owned

The screenshot shows a database management tool interface. On the left is a sidebar with a tree view of database objects. The 'Schemas (1)' folder is selected. The main area displays a SQL query and its results.

SQL Query:

```

1 SELECT m.title, a.category, a.award_year
2 FROM Award a
3 JOIN Movie m ON m.movie_id = a.movie_id
4 LEFT JOIN Inventory i ON i.movie_id = m.movie_id
5 WHERE a.award_name = 'Oscar'
6 GROUP BY m.title, a.category, a.award_year
7 HAVING COUNT(i.inventory_id) = 0
8 ORDER BY a.award_year DESC, m.title;

```

Data Output:

	title character varying (200)	category character varying (50)	award_year integer
1	Parasite	Best Picture	2020
2	Moonlight	Best Picture	2017
3	Casablanca	Best Picture	1944

Note. Query identifies movies with Academy Awards that Johnson Video does not currently own. Implementation joins Award -> Movie, filters award_name = 'Oscar', then LEFT JOINs Inventory and groups by movie, returning titles where COUNT(i.inventory_id) = 0. This measures ownership (physical copies in Inventory), not catalog availability. Award year denotes the ceremony year (which may differ from the movie's release year).

Which titles currently have only one copy in inventory?

Figure 16

Single-Copy Titles in Inventory

- > Schwenk_J_CT6
- > Schwenk_J_D5
- > Schwenk_J_D6
- > Schwenk_J_M3CT
- ▼ Schwenk_J_PP
 - > Casts
 - > Catalogs
 - > Event Triggers
 - > Extensions
 - > Foreign Data Wrappers
 - > Languages
 - > Publications
 - > Schemas (1)
 - > Subscriptions
 - > postgres
- > Login/Group Roles
- > Tablespaces

```

1  SELECT
2      m.title,
3      m.release_year,
4      COUNT(*) AS available_copies
5  FROM Inventory i
6  JOIN Movie m ON m.movie_id = i.movie_id
7  WHERE i.status = 'available'
8  GROUP BY m.movie_id, m.title, m.release_year
9  HAVING COUNT(*) = 1
10 ORDER BY m.title;

```

Data Output
Messages
Notifications

≡
📄
▼
📋
▼
🗑️
📦
⬇️
📈
SQL

	title character varying (200)	release_year integer	available_copies bigint
1	Argo	2012	1
2	Black Panther	2018	1
3	Gladiator	2000	1
4	Her	2013	1
5	Inception	2010	1
6	La La Land	2016	1
7	Pulp Fiction	1994	1
8	Spirited Away	2001	1
9	The Dark Knight	2008	1
10	The Godfather	1972	1
11	The King's Speech	2010	1
12	The Social Network	2010	1
13	Titanic	1997	1
14	Whiplash	2014	1

Note. Query identifies movie titles with exactly one copy in store inventory. Implementation joins Inventory to Movie, groups by movie, and filters with HAVING COUNT(*) = 1.

Rentals

Which rentals were returned late and fees collected?

Figure 17

Late Returns and Fees Collected

Schwenk_J_CT6

Schwenk_J_D5

Schwenk_J_D6

Schwenk_J_M3CT

Schwenk_J_PP

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas (1)

Subscriptions

postgres

Login/Group Roles

Tablespaces

1SELECT r.rental_id, c.first_name, c.last_name, m.title,
2r.due_date, r.return_date,
3(r.late_fee + r.damage_fee + r.rewind_fee) AS fees_collected,
4r.total_charge
5FROM Rental r
6JOIN Customer c ON c.customer_id = r.customer_id
7JOIN Inventory i ON i.inventory_id = r.inventory_id
8JOIN Movie m ON m.movie_id = i.movie_id
9WHERE r.return_date IS NOT NULL AND r.return_date > r.due_date
10ORDER BY r.return_date DESC, r.rental_id;

Data Output

Messages

Notifications

SQL

	rental_id integer	first_name character varying (50)	last_name character varying (50)	title character varying (200)	due_date date	return_date date	fees_collected numeric	total_charge numeric (7,2)
1	20	Customer	20	Her	2025-08-15	2025-08-17	2.50	6.50
2	18	Customer	02	The King's Speech	2025-08-14	2025-08-16	2.00	5.55
3	17	Customer	16	Black Panther	2025-08-13	2025-08-15	2.00	5.60
4	15	Customer	14	Whiplash	2025-08-12	2025-08-14	2.00	5.65
5	13	Customer	12	The Social Network	2025-08-11	2025-08-13	2.00	5.70
6	10	Customer	10	Mad Max: Fury Road	2025-08-10	2025-08-12	2.50	6.85
7	11	Customer	10	Mad Max: Fury Road	2025-08-10	2025-08-12	3.00	6.40
8	8	Customer	08	No Country for Old Men	2025-08-09	2025-08-11	2.00	5.70
9	6	Customer	06	La La Land	2025-08-08	2025-08-10	2.00	5.75
10	2	Customer	02	The Matrix	2025-08-06	2025-08-08	2.50	5.15

Note. Query lists rentals returned after the due date and the fees collected per transaction. It joins Rental -> Customer -> Inventory -> Movie, filters with `r.return_date IS NOT NULL AND r.return_date > r.due_date`, and computes `fees_collected` as `(r.late_fee + r.damage_fee + r.rewind_fee)` (while `total_charge` also includes tax). Results are ordered by most recent `return_date`.

Marketing

Who were the top renters in the last 30 days?

Figure 18

Top Renters in the Last 30 Days

Schwenk_J_CT6

Schwenk_J_D5

Schwenk_J_D6

Schwenk_J_M3CT

Schwenk_J_PP

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas (1)

Subscriptions

postgres

Login/Group Roles

Tablespaces

1 SELECT c.customer_id, c.first_name, c.last_name,
2 COUNT(*) AS rentals_30d,
3 SUM(r.total_charge) AS spend_30d
4 FROM Rental r
5 JOIN Customer c ON c.customer_id = r.customer_id
6 WHERE r.rental_date >= CURRENT_DATE - INTERVAL '30 days'
7 GROUP BY c.customer_id, c.first_name, c.last_name
8 ORDER BY rentals_30d DESC, spend_30d DESC, c.last_name;

Data Output

Messages

Notifications

≡

📄

▼

📄

▼

🗑️

🗑️

📥

📥

📈

SQL

	customer_id [PK] integer	first_name character varying (50)	last_name character varying (50)	rentals_30d bigint	spend_30d numeric
1	20	Customer	20	1	6.50
2	14	Customer	14	1	5.65
3	16	Customer	16	1	5.60
4	2	Customer	02	1	5.55
5	13	Customer	13	1	3.90
6	15	Customer	15	1	3.60
7	19	Customer	19	1	3.50

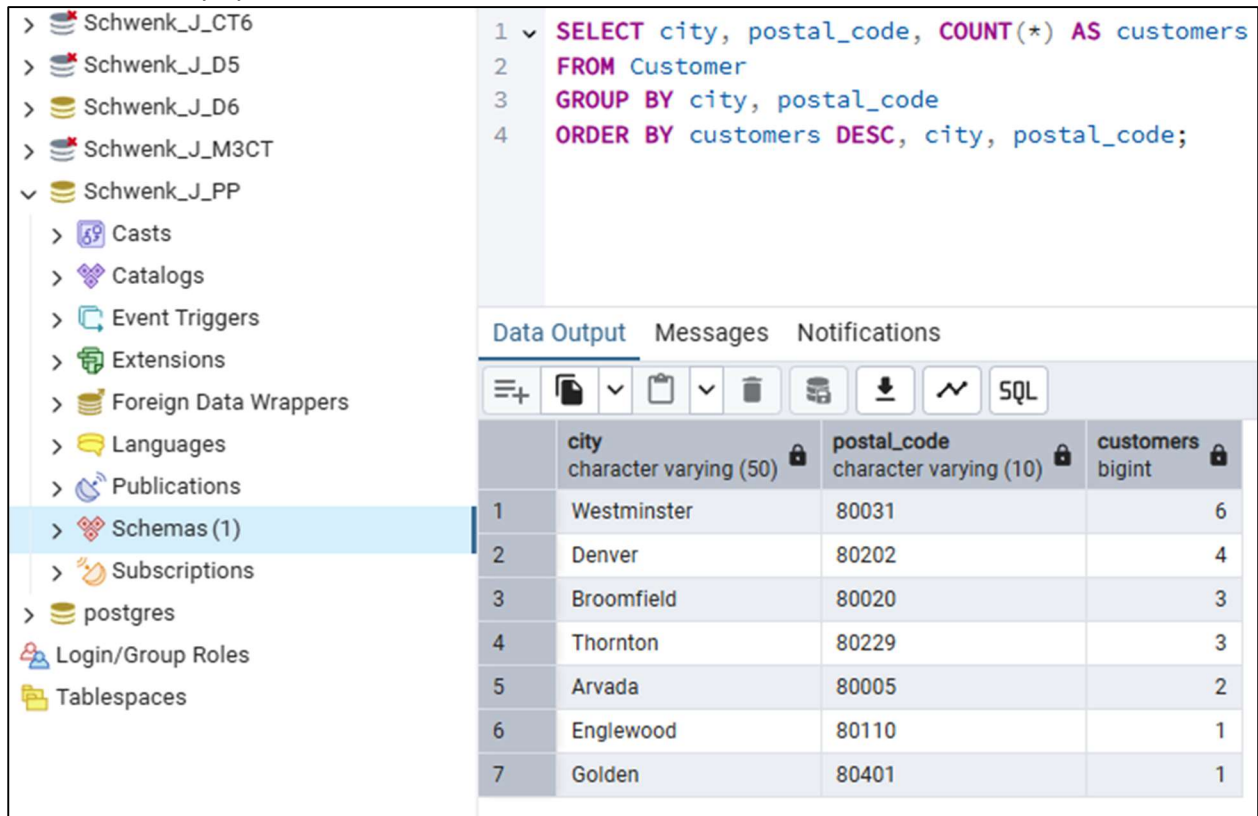
Note. Query ranks customers by rental volume over the past 30 days and breaks ties by spending. It joins Rental to Customer, filters on `rental_date >= CURRENT_DATE - INTERVAL '30 days'`, then aggregates

COUNT(*) AS rentals_30d and SUM(total_charge) AS spend_30d. Results are ordered by rentals_30d DESC, spend_30d DESC, last_name.

Which locations have the most customers?

Figure 19

Customer Density by Location



Note. Query counts customers by city + ZIP to pinpoint high-density pockets in the Denver metro. It aggregates COUNT(*) from Customer, groups by city, postal_code, and orders by customers DESC, city, postal_code.

What actors have the most credits among the titles we currently own?

Figure 20

Owned Actor Credit

- > Schwenk_J_CT6
- > Schwenk_J_D5
- > Schwenk_J_D6
- > Schwenk_J_M3CT
- ▼ Schwenk_J_PP
 - > Casts
 - > Catalogs
 - > Event Triggers
 - > Extensions
 - > Foreign Data Wrappers
 - > Languages
 - > Publications
 - > Schemas (1)
 - > Subscriptions
- > postgres
- > Login/Group Roles
- > Tablespaces

```

1 SELECT p.first_name, p.middle_initial, p.last_name,
2       COUNT(DISTINCT ma.movie_id) AS owned_credits
3 FROM MovieActor ma
4 JOIN Actor a ON a.actor_id = ma.actor_id
5 JOIN MoviePerson p ON p.person_id = a.actor_id
6 JOIN Inventory i ON i.movie_id = ma.movie_id
7 GROUP BY p.first_name, p.middle_initial, p.last_name
8 ORDER BY owned_credits DESC, p.last_name;

```

Data Output
Messages
Notifications

≡
📄
▼
🗑️
🔄
📥
📤
📶
SQL

	first_name character varying (50)	middle_initial character (1)	last_name character varying (50)	owned_credits bigint
1	Ben	[null]	Affleck	1
2	Christian	[null]	Bale	1
3	Javier	[null]	Bardem	1
4	Marlon	[null]	Brando	1
5	Russell	[null]	Crowe	1
6	Leonardo	[null]	DiCaprio	1
7	Jesse	[null]	Eisenberg	1
8	Colin	[null]	Firth	1
9	Rumi	[null]	Hiragi	1
10	Samuel	L	Jackson	1
11	Michael	B	Jordan	1
12	Joaquin	[null]	Phoenix	1
13	Keanu	[null]	Reeves	1
14	J.K.	[null]	Simmons	1
15	Emma	[null]	Stone	1
16	Charlize	[null]	Theron	1
17	Kate	[null]	Winslet	1

Note. Query ranks actors by the number of distinct internal movies we currently own. It joins MovieActor -> Actor -> MoviePerson and requires an Inventory row for each credited movie so only owned titles are counted. COUNT(DISTINCT ma.movie_id) prevents inflation from multiple copies of the same film. Results are ordered by owned_credits DESC.

Which people appear in the dataset as both actor and director?

Figure 21

Individuals Appearing as Both Actor & Director

Schwenk_J_CT6

Schwenk_J_D5

Schwenk_J_D6

Schwenk_J_M3CT

Schwenk_J_PP

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

1

SELECT p.person_id, p.first_name, p.middle_initial, p.last_name

2

FROM MoviePerson p

3

JOIN Actor a ON a.actor_id = p.person_id

4

JOIN Director d ON d.director_id = p.person_id

5

ORDER BY p.last_name, p.first_name;

Data Output

Messages

Notifications

person_id

[PK] integer

first_name

character varying (50)

middle_initial

character (1)

last_name

character varying (50)

1

15

Ben

[null]

Affleck

Note. Query returns people who exist in both subtype tables (Actor and Director) by joining them on the shared key from MoviePerson ($p.person_id = a.actor_id = d.director_id$). This identifies multi-role contributors in the dataset regardless of whether they acted and directed in the same film. Order by last/first name for readability; group by or select `DISTINCT p.person_id` to avoid duplicates if you later add role details. To restrict to cases where a person both acted and directed the same movie, extend with joins to MovieActor and MovieDirector and match $ma.movie_id = md.movie_id$.

Lessons Learned

Working end-to-end, from business rules to ERD, DDL, seed data, and queries, surfaced a handful of practical lessons that improved both the design and my knowledge.

- **SERIAL keys during seeding:** I learned while populating the database is the hassle of relying on SERIAL primary keys during seeding. Because their values are generated at the time of insertion, I couldn't know the IDs up front. This turned tedious as I found myself counting positions (e.g. If Quentin Tarantino was the 23rd MoviePerson, his `person_id` would be 23) when connecting rows across tables.
- **WITH Table Expression:** Complex queries became clearer when I staged them. For example, I built a price candidate set in a common expression table (CTE) and then selected the cheapest row.
- **LEFT JOIN semantics:** Several business questions required keeping unmatched rows. A key lesson was to place filters that would null-out the outer rows in the JOIN clause instead of the WHERE clause, which can accidentally turn the join into an INNER JOIN and drop the vendors I was trying to find.
- **Window Functions and PARTITION BY:** Ranking and picking the minimum per group are naturally solved with `ROW_NUMBER() OVER (PARTITION BY movie, format ORDER BY price)`. In PostgreSQL, filtering the winner is done with an outer query (`WHERE rn = 1`). Window functions avoided over-aggregation and let me keep detail columns while still finding the best prices.
- **DATE math with INTERVAL:** I standardized on `CURRENT_DATE - INTERVAL '30 days'` for windows. Being explicit about whether a metric keys off `rental_date` or `return_date` prevented ambiguity.
- **Seed data shape insights:** What you seed determines what you can learn. My set every `Inventory.status` to "available", which made status analytics meaningless. Ultimately, I had to change the question.

References

- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Pearson Education.
- Mannino, M. (2022). *Database design: Design, query, formulation, and administration using Oracle and PostgreSQL*. SAGE Publications.
- Stair, R., & Reynolds, G. (2018). *Fundamentals of information systems* (9th ed.). Cengage.
- Valacich, J. S., & George, J. F. (2017). *Modern systems analysis and design* (9th ed.). Pearson Education.

Appendix

<refer to any file attachments>